

Linear Transformations and Matrices

Contents

1. Overview	2
2. Transformations: functions that move vectors	3
3. What makes a transformation linear	3
4. Two vectors decide everything	3
5. The matrix of a transformation	4
6. Reading a matrix as a picture	5
7. Where ML uses linear transformations	7
7.1 One matrix for the whole dataset	7
7.2 Feature scaling is a stretch along the axes	8
7.3 Neural network layers and PCA	8
8. Summary	9
9. Key terms	9

Where this fits: Pipeline map step 0 (Foundations). See the [Course map](#).



- **Builds on:** Linear combinations, span and basis ([Note 490](#)).
- **Leads to:** Matrix multiplication as composition ([Note 510](#)); Eigenvectors and eigenvalues ([Note 530](#)); Determinant ([Note 530](#)).

1. Overview

Key point: A matrix is a record of where the basis vectors land under a linear transformation; multiplying a matrix by a vector computes where that vector lands.

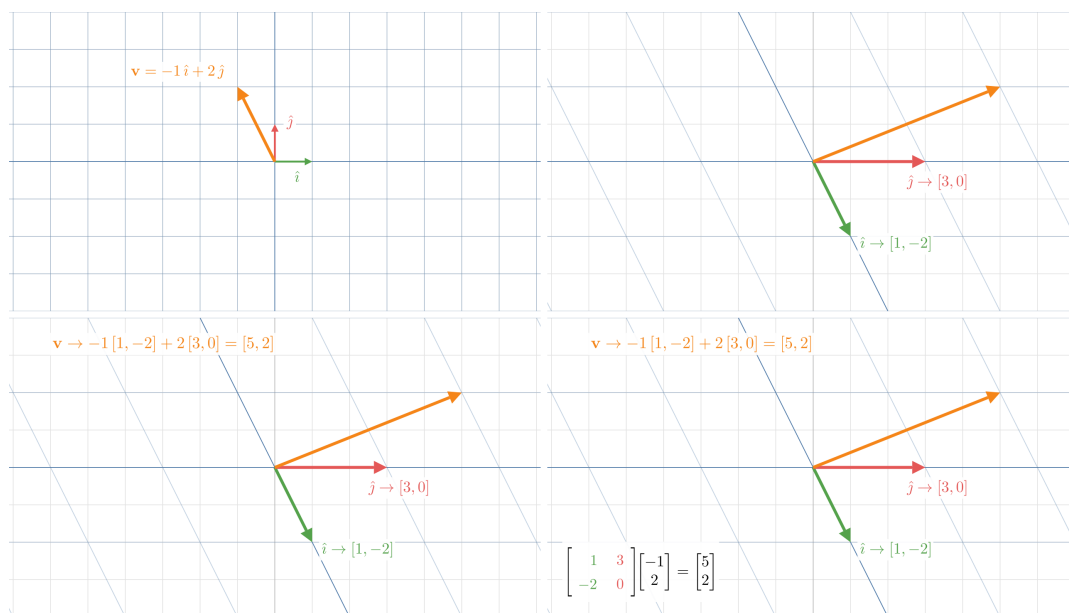


Figure 1: A linear transformation moves the whole grid. Where $\hat{i}$ (green) and $\hat{j}$ (red) land decides where every other vector, such as $\mathbf{v}$ (orange), lands

Figure 1 shows the idea of this Note. The whole plane moves: grid lines stay straight, parallel and evenly spaced, and the origin stays put. We only need to know where the two basis vectors $\hat{i}$ and $\hat{j}$ land; those four numbers, written as the columns of a matrix, tell us where every other vector goes.

The [PCA step by step Note](#) (section 4.1) met this idea in one paragraph: a matrix as an action on the whole plane. This Note explains why it works and how to read any matrix that way. It uses the basis vectors, linear combinations and span of the [linear combinations, span and basis Note](#).

2. Transformations: functions that move vectors

Key point: A transformation takes a vector in and gives a vector out; we picture it as every point of space moving to its new place.

A **transformation** is a function whose input is a vector and whose output is another vector. The word “transformation” suggests movement: we imagine each input vector moving to its output vector.

To see a transformation as a whole, we watch every vector move at once. Drawing every vector as an arrow would be far too crowded. So we draw each vector as the point at its tip, and watch all the points of a grid move. A faint copy of the original grid stays in the background, so we can see where things went (the grey grid in Figure 1).

3. What makes a transformation linear

Key point: A transformation is linear when all lines stay straight lines and the origin stays fixed; then grid lines stay parallel and evenly spaced.

General transformations can bend space in complicated ways. Linear algebra studies only the simple kind. Visually, a transformation is **linear** when it has two properties:

- **Lines stay lines.** No straight line gets curved.
- **The origin stays fixed.**

Together these mean that grid lines stay parallel and evenly spaced. Some examples that fail:

- A transformation that curves the grid lines is not linear.
- A transformation that keeps lines straight but slides everything to the right moves the origin, so it is not linear.
- A transformation can keep the horizontal and vertical grid lines straight and still bend a diagonal line. It is not linear either: the test applies to every line.

Rotations about the origin, stretches along the axes and the slanting of Figure 1 are linear.

Extra: The formal definition says the same thing with algebra. A transformation L is linear when it respects the two operations of linear algebra: $L(\mathbf{v} + \mathbf{w}) = L(\mathbf{v}) + L(\mathbf{w})$ and $L(c\mathbf{v}) = cL(\mathbf{v})$ for every scalar c . Section 4 is exactly these two rules in action.

4. Two vectors decide everything

Key point: A vector that starts as $x\hat{i} + y\hat{j}$ lands on x times where $\hat{i}$ lands plus y times where $\hat{j}$ lands.

How can we describe a linear transformation with numbers, so that a computer can tell where any vector lands? We only need to record where $\hat{i}$ and $\hat{j}$ land.

Take $\mathbf{v} = [-1, 2]$, which is $-1\hat{i} + 2\hat{j}$. Because grid lines stay parallel and evenly spaced, $\mathbf{v}$ lands on the same linear combination of the new $\hat{i}$ and the new $\hat{j}$.

1. **In words:** keep the coordinates of the vector, but use them to scale the landed basis vectors instead of the original ones.
2. **Formula:** if $\hat{i}$ lands on $\mathbf{i}'$ and $\hat{j}$ lands on $\mathbf{j}'$,

$$x \hat{i} + y \hat{j} \longrightarrow x \mathbf{i}' + y \mathbf{j}'$$

3. **Example:** in Figure 1, $\hat{i}$ lands on $[1, -2]$ and $\hat{j}$ on $[3, 0]$. So

$$[-1, 2] \longrightarrow -1[1, -2] + 2[3, 0] = [-1 + 6, 2 + 0] = [5, 2]$$

We did not have to watch the animation to know that $\mathbf{v}$ lands on $[5, 2]$. For a general vector $[x, y]$ the same reasoning gives

$$x[1, -2] + y[3, 0] = [x + 3y, -2x]$$

a formula for where every vector lands. So a 2D linear transformation is completely described by four numbers: the two coordinates where $\hat{i}$ lands and the two where $\hat{j}$ lands.

5. The matrix of a transformation

Key point: Write where $\hat{i}$ lands as the first column and where $\hat{j}$ lands as the second; that 2×2 grid of numbers is the matrix of the transformation.

We package the four numbers into a 2×2 **matrix** whose columns are the landed basis vectors:

$$A = \begin{bmatrix} 1 & 3 \\ -2 & 0 \end{bmatrix} \quad \text{first column: where } \hat{i} \text{ lands; second column: where } \hat{j} \text{ lands}$$

Matrix-vector multiplication $A\mathbf{x}$ is the computation that applies the transformation to the vector $\mathbf{x}$: scale each column by the matching coordinate of $\mathbf{x}$ and add.

1. **In words:** multiply the first column by the first coordinate, the second column by the second coordinate, and add the two.
2. **Formula:** for a general matrix,

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = x \begin{bmatrix} a \\ c \end{bmatrix} + y \begin{bmatrix} b \\ d \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

3. **Example:**

$$\begin{bmatrix} 1 & 3 \\ -2 & 0 \end{bmatrix} \begin{bmatrix} -1 \\ 2 \end{bmatrix} = -1 \begin{bmatrix} 1 \\ -2 \end{bmatrix} + 2 \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$$

The formula on the right, $[ax + by, cx + dy]$, is the rule usually memorised in school. The middle form says what it means: the result is a linear combination of the columns, with the vector's coordinates as the scalars. The matrix is written to the left of the vector, like a function written to the left of its input.

Python: In NumPy the @ operator multiplies a matrix by a vector.

```
import numpy as np
```

```
A = np.array([[1, 3], [-2, 0]])
```

```
A @ np.array([-1, 2])      # array([5, 2])
```

```
A[:, 0], A[:, 1]          # where  $\hat{i}$  and  $\hat{j}$  land
```

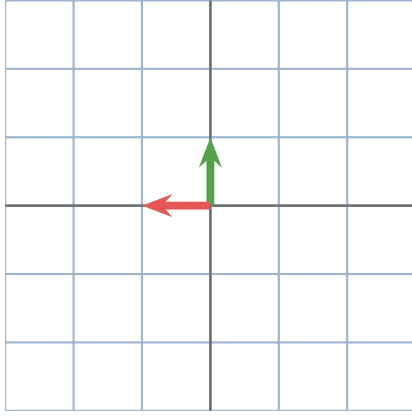
```
A[:, 0] takes all rows of column 0: the first column.
```

6. Reading a matrix as a picture

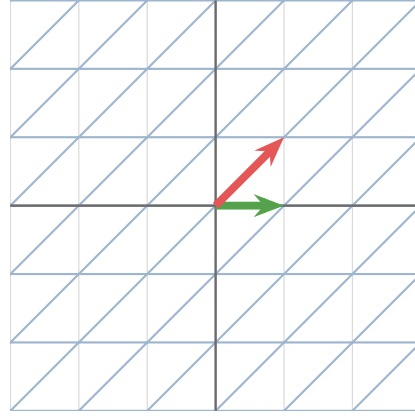
Key point: Read the columns, move $\hat{i}$ and $\hat{j}$ there, and let the rest of the grid follow.

Every matrix can be read as a transformation, and every linear transformation has a matrix. Figure 2 shows four of them; in each, green is where $\hat{i}$ lands and red is where $\hat{j}$ lands.

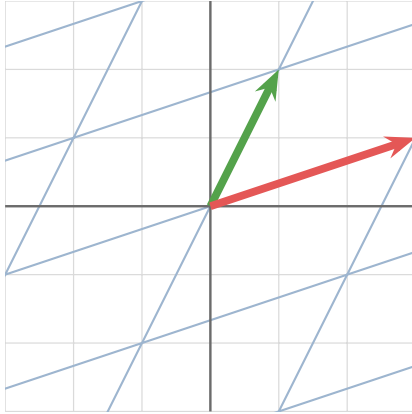
Rotation by 90° : $\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$



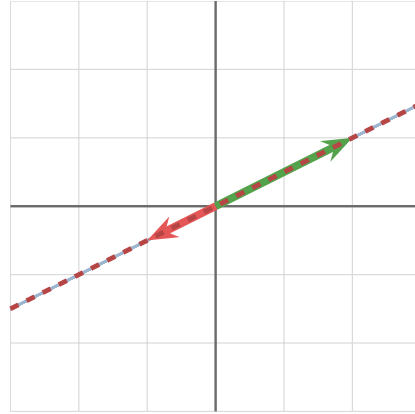
Shear: $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$



$\begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix}$: $\hat{i} \rightarrow [1, 2]$, $\hat{j} \rightarrow [3, 1]$



Dependent columns: $\begin{bmatrix} 2 & -1 \\ 1 & -0.5 \end{bmatrix}$



the whole plane is squished onto one line

green: where $\hat{i}$ lands (first column) red: where $\hat{j}$ lands (second column) grey: the grid before

Figure 2: Four matrices read as transformations of the grid

- **Rotation by 90° counterclockwise.** $\hat{i}$ lands on $[0, 1]$ and $\hat{j}$ on $[-1, 0]$, so the matrix has columns $[0, 1]$ and $[-1, 0]$. Any vector rotates by multiplying with it: $[3, 1]$ lands on $[-1, 3]$.
- **Shear.** $\hat{i}$ stays at $[1, 0]$ and $\hat{j}$ moves to $[1, 1]$. Horizontal lines slide sideways, more the higher they are: $[2, 3]$ lands on $[5, 3]$.
- **From matrix to picture.** For columns $[1, 2]$ and $[3, 1]$, we move $\hat{i}$ to $[1, 2]$ and $\hat{j}$ to $[3, 1]$ and keep the grid lines parallel and evenly spaced. The vector $[1, 1]$ lands on $[1, 2] + [3, 1] = [4, 3]$.
- **Squishing onto a line.** If the two columns are linearly dependent, here $[-1, -0.5] =$

$-0.5 \times [2, 1]$, the whole plane is squished onto the line they span. $[1, 1]$ lands on $[1, 0.5]$ and $[3, -1]$ on $[7, 3.5]$: both on the line through $[2, 1]$.

The last case links back to the [linear combinations, span and basis Note](#): every output is a linear combination of the columns, so all outputs lie in the span of the columns. Independent columns span the plane; dependent ones only a line.

Extra: The span of the columns is called the **column space** of the matrix, one of the topics the [linear algebra roadmap Note](#) lists. Its dimension is the rank of the matrix: 2 for the first three matrices of Figure 2, 1 for the squishing one.

7. Where ML uses linear transformations

Key point: Feature scaling, PCA and every layer of a neural network apply a matrix to each data point.

Once a matrix is a transformation, many ML steps become pictures of moving space.

7.1 One matrix for the whole dataset

Key point: With the data points as the rows of X , the product XA^T applies A to every row at once.

A dataset is a stack of feature vectors, one per row: the **data matrix** X (the design matrix of the [multiple linear regression maths Note](#), without the column of ones). To transform every point we could loop over the rows and compute $A\mathbf{x}$ for each. NumPy does all of them in one product.

1. **In words:** each row of the result is the transformed version of that row of X .
2. **Formula:** for n points with d features (X is $n \times d$),

$$\text{new data} = XA^T$$

The transpose appears because the points are stored as rows, while $A\mathbf{x}$ treats a point as a column.

3. **Example:** with A from Section 5 and the three points $[-1, 2]$, $[1, 0]$ and $[0, 1]$ as rows,

$$\begin{bmatrix} -1 & 2 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ 3 & 0 \end{bmatrix} = \begin{bmatrix} 5 & 2 \\ 1 & -2 \\ 3 & 0 \end{bmatrix}$$

The second and third rows are where $\hat{i}$ and $\hat{j}$ land, as they should be.

Python:

```
X = np.array([[ -1,  2], [ 1,  0], [ 0,  1]])
X @ A.T      # [[ 5,  2], [ 1, -2], [ 3,  0]]
```

7.2 Feature scaling is a stretch along the axes

Key point: Dividing each column by a number is a diagonal matrix: it stretches or squishes space along the axes only.

Dividing a column by its standard deviation, the second half of [standardization](#), is a linear transformation. With standard deviations 2 and 10, $\hat{i}$ lands on $[0.5, 0]$ and $\hat{j}$ on $[0, 0.1]$:

$$\begin{bmatrix} 0.5 & 0 \\ 0 & 0.1 \end{bmatrix}$$

A matrix with zeros everywhere off the diagonal is a **diagonal matrix**. Its picture is simple: each axis is stretched or squished by its own factor, and nothing rotates or slants.

7.3 Neural network layers and PCA

Key point: A neural network layer multiplies by a weight matrix, adds a shift, then bends the result; PCA multiplies by a matrix of eigenvectors.

- **A neural network layer** computes $W\mathbf{x} + \mathbf{b}$ and then applies an activation function such as the [sigmoid](#). $W\mathbf{x}$ is a linear transformation of the input. Adding $\mathbf{b}$ moves the origin, and the activation bends the lines, so the layer as a whole is not linear. Those two extra steps are what let a network learn curved boundaries.
- **PCA** projects each point with $Z = XW^T$ (see the [PCA step by step Note](#), section 5.1): a matrix applied to every row, exactly as in Section 7.1.

Extra: A transformation followed by a shift, $A\mathbf{x} + \mathbf{b}$, is called an **affine transformation**. It keeps lines straight and parallel but moves the origin. Mean centring followed by scaling (standardization) is affine, and so is a neural network layer before its activation.

Extra: Matrices need not be square. A matrix with 2 rows and 3 columns has three columns, one for where each of the three 3D basis vectors lands, and each column has 2 numbers: it takes 3D vectors to 2D vectors. PCA from 3 features to 2 components is such a 2×3 matrix W . Its rows are the two eigenvectors; reading it by columns, each column says where one original axis lands in the new 2D picture.

8. Summary

Matrix	Where $\hat{i}$ lands	Where $\hat{j}$ lands	Effect
$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$	$[0, 1]$	$[-1, 0]$	rotation by 90°
$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$	$[1, 0]$	$[1, 1]$	shear
$\begin{bmatrix} 0.5 & 0 \\ 0 & 0.1 \end{bmatrix}$	$[0.5, 0]$	$[0, 0.1]$	scaling along the axes
$\begin{bmatrix} 2 & -1 \\ 1 & -0.5 \end{bmatrix}$	$[2, 1]$	$[-1, -0.5]$	squishes the plane onto a line

- A linear transformation keeps lines straight and the origin fixed; grid lines stay parallel and evenly spaced.
- It is decided by where the basis vectors land; those landing spots are the columns of its matrix.
- $A\mathbf{x}$ is a linear combination of the columns of A , with the coordinates of $\mathbf{x}$ as scalars.
- Every matrix is a transformation of space; XA^T applies it to a whole dataset.

9. Key terms

Term	Meaning
Transformation	A function that takes a vector in and gives a vector out
Matrix (of a transformation)	The grid of numbers whose columns are where the basis vectors land
Matrix-vector multiplication	$A\mathbf{x}$: the linear combination of the columns of A with the coordinates of $\mathbf{x}$ as scalars
Rotation matrix	The matrix of a rotation about the origin; by 90° , columns $[0, 1]$ and $[-1, 0]$
Shear	A transformation that keeps $\hat{i}$ fixed and slants $\hat{j}$, sliding horizontal lines sideways
Column space	The span of the columns of a matrix: every output it can produce
Data matrix	The feature vectors of a dataset stacked as rows
Diagonal matrix	A matrix with zeros everywhere off the diagonal; it scales each axis by its own factor
Affine transformation	A linear transformation followed by a shift, $A\mathbf{x} + \mathbf{b}$